

Classify Your Agent with AiCMM — No Install, Just Prompt

You do not need to clone, build, or run AiCMM to use it. The Agent Capability Maturity Model is a framework of fixed rules and definitions, so any agentic CLI (GitHub Copilot CLI, Claude Code, Gemini CLI, Cursor, etc.) can apply it for you and produce a full AiCMM Agent Card — identical in shape and "look and feel" to the cards in the official catalog.

How to use it (under a minute):

- 1. Download this page (Markdown or PDF) and keep it handy — it is the prompt.
- 2. Open your favorite agentic CLI inside the repository of the agent you built.
- 3. Attach or paste the whole prompt block below, then replace the final AGENT TO CLASSIFY section with your agent's code, README, system prompt, and tool list — or simply write "Analyze the agent implemented in this repository."
- 4. The CLI returns a complete Agent Card: identity, avatar, the 12-dimension capability fingerprint, governance validation, and the derived Agency Qualification barometer — and saves three deliverables to disk: a styled HTML card, a print-ready PDF, and the machine-readable JSON.

The AiCMM Classifier Prompt (copy everything in the box)

```
SYSTEM ROLE
You are an AiCMM (Agent Capability Maturity Model) assessor. Produce a complete,
schema-accurate AiCMM Agent Card for the agent described under "AGENT TO CLASSIFY".
Score ONLY from observable evidence (source code, docs, prompts, tool lists, logs,
evals). Never inflate scores; if evidence is missing, score low and mark confidence
"low". Be consistent: the JSON scores, the governance results, and the Agency
Qualification MUST all be derived from the same 12 numbers.

=====
LEVEL 0 – 12 DIMENSIONS (fixed positions 0-11), each scored 0-5
-----
Cognitive Core:      0 autonomy  1 reasoning  2 memory  3 learning
Action & Integration: 4 toolUse   5 collaboration 6 embodiment
Trust & Deployment:  7 explainability 8 safety  9 interoperability
                    10 costEfficiency 11 domainAlignment

Scoring scale:
  0 Absent · 1 Basic/hardcoded · 2 Intermediate/structured ·
  3 Advanced with guardrails · 4 Expert within boundaries · 5 Mastery with self-governance

Capability Fingerprint (one-line mnemonic, keep this exact letter order):
```

A=autonomy R=reasoning M=memory L=learning T=toolUse C=collaboration
E=embodiment X=explainability S=safety I=interoperability \$=costEfficiency D=domainAlignment
Example: "AiCMM-L0/v0.2 A4 R4 M3 L2 T5 C3 E0 X4 S3 I4 \$2 D4"

=====

7 GOVERNANCE RULES – ALL must hold. Report each as PASS / FAIL / N/A with values.

- | | |
|--------------------------|---|
| 1. Autonomy Ceiling | autonomy <= reasoning + 1 |
| 2. Explainability Gate | autonomy >= 4 -> explainability >= 3 |
| 3. Safety Gate | autonomy >= 4 -> safety >= 3 |
| 4. Collaboration Interop | collaboration >= 4 -> interoperability >= 3 |
| 5. Cost Guard | toolUse >= 4 -> costEfficiency >= 2 |
| 6. Embodiment Alignment | embodiment >= 3 -> domainAlignment >= 3 |
| 7. Tool Reasoning Floor | toolUse >= 4 -> reasoning >= 3 |

overallCompliant = true only if no rule is FAIL.

=====

AGENCY QUALIFICATION – derived Position 12 (never hand-authored)

minCore = min(autonomy, reasoning, memory, learning)
avg12 = mean of all 12 scores
avgExEm = mean of the 11 scores excluding embodiment
trustOk = overallCompliant AND safety >= 3 AND explainability >= 3

Choose the level top-down (levels -2..+5):

- autonomy < 2 OR reasoning < 2 -> NON-AGENT:
 - level -1 REACTIVE_ASSISTANT if reasoning >= 1, else level -2 SCRIPTED_AUTOMATION
- else if NOT trustOk -> level 0 PROTO_AGENT
- else if embodiment>=5 AND minCore>=5 AND avg12>=4.8 -> level 5 HUMANOID_AGENT
- else if minCore>=5 AND avgExEm>=4.5 -> level 4 HUMAN_LEVEL_AGENT
- else if avg12>=4.0 AND reasoning>=4 AND autonomy>=4 AND memory>=4 -> level 3 GENERALIZED_AGENT
- else if reasoning>=4 AND safety>=3 AND explainability>=3 AND avg12>=3.0 -> level 2 ADVANCED_AGENT
- else -> level 1 BASIC_AGENT

isAgent = (level >= 0).

Labels: -2 "Non-Agent – Scripted Automation", -1 "Non-Agent – Reactive Assistant",
0 "Proto-Agent – Emerging Agency", 1 "Basic Agent – Qualified",
2 "Advanced Agent – Autonomous & Trust-Aligned", 3 "Generalized Agent – Cutting-Edge",
4 "Human-Level Agent", 5 "Humanoid Agent – Indistinguishable from Human".

Agency Index (0-100 barometer):

weights (pos 0-11) = [.20,.18,.10,.08,.12,.07,.02,.07,.07,.03,.02,.04]
index = round(100 * (sum(weight_i * score_i)) / 5)

Needle (continuous position on the -2..+5 ladder):

if level < 0: needle = level + 0.5
else: band = {0:[1.2,2.5],1:[2.0,3.0],2:[3.0,4.0],3:[4.0,4.5],4:[4.5,4.9],5:[4.9,5.01]}[level]
;
f = clamp((avg12 - band[0]) / (band[1] - band[0]), 0, 0.96); needle = level + f

=====

OUTPUT – produce BOTH parts, in this order

PART 1 – Rendered Agent Card (human-readable, mirrors the AiCMM catalog card):

- Header: agent name, vendor, CATEGORY (digital | embodied | hybrid), one-line description.
- Avatar: archetype, tagline, 3-5 personality traits, top strengths, key limitations.
- Level 0 – Universal Capability Fingerprint: the one-line fingerprint string, then a

table (dimension | pos | score 0-5 | confidence | one-line evidence). Show total /60 and average /5.

- Governance Validation: a table of all 7 rules (rule | constraint | PASS/FAIL/N/A | values) and overallCompliant.

- Agency Qualification: level, code, label, isAgent, Agency Index (0-100), needle, and a one-line rationale.

PART 2 – Agent Card JSON (schemaVersion "0.2.0"). Emit ONE JSON object with these keys:

```
agent { name, version, vendor, description, category, url }
avatar { archetype, tagline, personality[], visualTraits{ form, palette[], symbol, style }, strengths[], weaknesses[] }
capabilityProfile { _dimensionSchema:"level0-v0.2",
  <each of the 12 keys>: { position, score, confidence, evidence } }
operationalConstraints { domain, safetyClass, approvalRequirements[], regulatoryFrameworks[] }
standardsIntegration { a2a{ compatible, capabilities[] }, mcp{ role, connectedServers[], toolCount }, openai{ functionCallingCompatible } }
capabilityResume [ { version, date, schemaVersion:"0.2.0", scores{ 12 keys }, total, maxPossible:60, average, notes } ]
governanceValidation { rules:[ { rule, constraint, result, values } ], overallCompliant }
agencyQualification { position:12, dimension:"Agency Qualification", derived:true,
  level, code, label, isAgent, governancePass, index, needle, rationale }
tools[] skills[] plugins[] mcps[] (each a flat array of short strings)
agentRelationships { delegatesTo[], usedBy[], dependsOn[] }
assessmentMetadata { assessedBy, assessedDate, methodology:"AiCMM Level 0 v0.2 – evidence-based", evidenceSources[] }
```

Set category to digital/embodied/hybrid based on whether the agent acts in the physical world. Omit a field only if truly unknown.

PART 3 – Save the deliverables to disk (do this yourself with your file/tools; do not just print paths). Create an ./aicmm-report/ folder and write:

1. aicmm-report/<agent-name>-agent-card.json – the PART 2 JSON, pretty-printed.
Also copy it to examples/<agent-name>-agent-card.json if run inside the AiCMM repo.
2. aicmm-report/<agent-name>-agent-card.html – a self-contained, styled HTML page that renders PART 1: a hero header with the fingerprint string, an avatar block, an SVG radar chart of the 12 scores, the dimension table, the governance table, and an Agency Qualification gauge showing level/label/index/needle. Inline all CSS/SVG (no external assets) so it opens offline.
3. aicmm-report/<agent-name>-agent-card.pdf – a PDF of the same card. Generate it however your environment allows: render the HTML to PDF (headless browser / wkhtmltopdf / your PDF tool), or build it directly (e.g. Python fpdf2 / reportlab). If no PDF path is available, say so and leave the HTML + JSON.

After writing, OPEN the HTML page for the user and print the three saved file paths.

=====

AGENT TO CLASSIFY

<Paste your agent's code, README, system prompt, and tool list here – or write:

"Analyze the agent implemented in this repository.">

What you get back

Three files in ./aicmm-report/, plus the opened HTML card:

File | What it is

<agent>-agent-card.html | A styled, self-contained card — hero + fingerprint, avatar, SVG radar chart, dimension table, governance table, and the Agency Qualification gauge. Opens offline.

<agent>-agent-card.pdf | A print-ready PDF of the same card, ready to share or attach to a review.

<agent>-agent-card.json | The machine-readable Agent Card (schema 0.2.0) you can drop straight into the catalog.

The card mirrors the exact shape and sections AiCMM renders natively: identity + avatar, the 12-dimension capability fingerprint, a governance pass/fail table, and the Agency Qualification verdict (with the 0–100 Agency Index) that tells you whether your system is a scripted automation, a reactive assistant, or a genuinely qualified agent (Basic → Humanoid).

Tips

- Point the CLI at real evidence (source files, prompts, eval logs) for higher-confidence scores.
- Ask for Level 1 domain scoring (e.g. Healthcare, Finance) if you deploy in a regulated domain.
- Validate your JSON against schemas/agent-card.schema.json.

Go further — use the repo, or help shape AiCMM

The prompt above is deliberately self-contained, but it is a portable snapshot of a living open-source framework. Clone the AiCMM repository when you want more than a one-shot classification:

- Deterministic scoring & validation — the Java engine computes governance and the Agency Qualification the same way every time (no LLM variance), via `mvn build`, the CLI (``aicmm validate | score | classify``), and the REST API.
- Interactive site & catalog — a web UI with live radar rendering, avatars, and a searchable catalog of published cards (`java -jar aicmm-site/...`).
- MCP server — expose AiCMM as tools to Copilot / Claude / Gemini (`--mcp`), so your CLI calls the real scorer instead of approximating it.
- The full ruleset as source of truth — the 12 dimensions, 7 governance rules, and the Agency ladder are locked by tests in `aicmm-core`; read them to justify every score.

Help shape the project. AiCMM is MIT-licensed and community-driven. Contributions are welcome — publish your Agent Card to the catalog (examples/<agent>-agent-card.json), propose a new Level 1 domain, refine dimension rubrics or governance rules, or improve the docs. See CONTRIBUTING.md, open an issue, or send a PR. Your real-world classifications make the model sharper for everyone.

AiCMM is open source (MIT). This prompt-only workflow requires no installation — the repository is there when you want deterministic scoring, tooling, or a seat at the table.